

BFAT Filesystem Performance Evaluation Report

Project: BFAT Filesystem

Disk Size: 16 MB (4096 blocks × 4KB)

1. Objective

This report evaluated the performance of the BFAT filesystem as it was described, by conducting a series of realistic file system operations. The main aim was to assess BFAT's performance in file creation, reading, writing, and deletion under typical usage scenarios. I created a script file to run these tests, I will be attaching that along with the code file if needed.

2. System Setup

The BFAT filesystem is implemented using FUSE version 2 in C. Testing was conducted inside a VirtualBox Ubuntu environment. A virtual disk of 16MB was created and formatted. Filesystem operations were measured using '/usr/bin/time -v' for system-level metrics.

Key configuration:

- Disk file: disk1 (16MB, 4096 blocks)
- Block size: 4096 bytes
- FAT length: 4 blocks
- Root directory block: 5
- Mount point: ~/bfat_mount

3. Experiments Conducted

Test 1: Create 30 Small Files

Command: `for i in {1..30}; do echo "data $i" > file_$i; done`

Result: Elapsed Time: 0.10s, System Time: 0.04s

Interpretation: Creation was efficient, with low overhead for directory and FAT updates. Confirms correct handling of single-block file creation.

Test 2: Write a 5MB File

Command: `dd if=/dev/zero of=bigfile bs=1M count=5`

Result: Elapsed Time: 0.59s, System Time: 0.16s

Interpretation: Demonstrates correct FAT chaining across multiple blocks. Write time increases due to sequential block updates.

Test 3: Read the 5MB File

Command: `cat bigfile > /dev/null`

Result: Elapsed Time: 0.00s, System Time: 0.00s

Interpretation: Extremely fast read access, indicating minimal read overhead and efficient FAT traversal.

Test 4: Delete 15 Files

Command: `rm file_{1..15}`

Result: Elapsed Time: 0.04s, System Time: 0.01s

Interpretation: Deletion was swift and confirmed proper FAT cleanup and directory slot reuse.

4. Conclusion

The BFAT filesystem performs very effectively for small and moderately sized files. It handles file creation, FAT chaining, reading, and deletion within the expected performance profile for a simplified filesystem.

Small file operations were nearly instant. Writing larger files was handled properly using FAT chaining logic. Read operations were also highly efficient. Deletion of files also successfully reclaimed directory and FAT resources.

Overall, BFAT meets the objectives for correctness and performance within the constraints of a 16MB disk.